

Lambda-Ausdrücke und Streams

Aspekt	Lambda-Ausdrücke	Streams
Definition	Ein Lambda-Ausdruck ist eine anonyme Funktion, die ein Ziel für eine Schnittstelle mit einer einzelnen abstrakten Methode (Funktionales Interface) darstellt. Er wird verwendet, um Funktionen als Parameter zu übergeben oder in einer klaren, prägnanten Weise zu definieren.	Ein Stream ist eine Folge von Daten, die über eine Pipeline von Operationen verarbeitet werden können. Streams sind sequenziell oder parallel und bieten ein funktionales Paradigma für die Verarbeitung von Daten.
Verwendungszweck	Lambda-Ausdrücke werden verwendet, um Funktionen auf eine kurze, prägnante Weise zu definieren und als Argumente für Methoden zu übergeben. Sie sind besonders nützlich in der funktionalen Programmierung.	Streams werden verwendet, um Daten auf eine deklarative Weise zu verarbeiten, indem sie Operationen wie Filterung, Mapping und Reduktion auf eine Datenquelle anwenden.
Syntax	<code>(parameters) -> expression</code> z.B. <code>x -> x * x</code> (nimmt <code>x</code> und gibt <code>x^2</code> zurück).	Ein Stream wird durch <code>Stream.of()</code> oder eine Sammlung wie <code>List.stream()</code> erzeugt. Die Stream-Operationen werden in einer Pipeline ausgeführt.
Beispiel	Beispiel eines Lambda-Ausdrucks: <code>java (a, b) -> a + b</code> ; Dies ist eine Funktion, die zwei Werte <code>a</code> und <code>b</code> addiert.	Beispiel eines Streams: <code>java List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5); numbers.stream().filter(n -> n % 2 == 0).forEach(System.out::println);</code> Filtert gerade Zahlen und gibt sie aus.
Ziel	Der Lambda-Ausdruck wird verwendet, um eine Funktion oder Aktion zu definieren und zu übergeben.	Ein Stream ermöglicht eine sequentielle oder parallele Verarbeitung von Daten, die durch verschiedene Operationen wie <code>filter()</code> , <code>map()</code> und <code>reduce()</code> transformiert werden können.
Parameter	Lambda-Ausdrücke können beliebig viele Parameter annehmen, aber müssen immer einen Rückgabewert liefern.	Ein Stream kann jeden Datentyp enthalten (Objekte, primitive Typen). Die Stream-Operationen werden auf den Daten im Stream angewendet.
Rückgabewert	Ein Lambda-Ausdruck gibt einen Wert oder eine Aktion zurück, der/die im Kontext der Funktionalität verwendet wird.	Ein Stream gibt eine modifizierte Datenquelle zurück (z.B. eine gefilterte Liste). Der ursprüngliche Stream wird dabei nicht verändert, sondern eine neue Stream-Instanz wird zurückgegeben.

Aspekt	Lambda-Ausdrücke	Streams
Funktionale Interfaces	Lambda-Ausdrücke werden hauptsächlich für funktionale Interfaces verwendet, die nur eine abstrakte Methode definieren. Beispiele sind: <code>Runnable</code> , <code>Comparator</code> , <code>Function</code> , <code>Predicate</code> .	Streams verwenden keine speziellen funktionalen Interfaces, jedoch werden häufig die funktionalen Interfaces wie <code>Function</code> , <code>Predicate</code> , <code>Consumer</code> und <code>Supplier</code> für Operationen wie <code>map()</code> , <code>filter()</code> , <code>forEach()</code> verwendet.
Verwendung mit Sammlungen	Lambda-Ausdrücke können als Argumente für Methoden in Sammlungen verwendet werden, um Operationen wie Sortieren oder Filtern zu definieren.	Streams können von Sammlungen durch die <code>stream()</code> -Methode erzeugt werden, die eine sequenzielle oder parallele Verarbeitung der Elemente ermöglicht.
Modifikation des Zustands	Lambda-Ausdrücke können den Zustand von Objekten oder Variablen ändern, aber der Zustand sollte vorsichtig verwaltet werden, um nebenläufige Probleme zu vermeiden.	Streams sind in der Regel unveränderlich. Jede Operation erzeugt einen neuen Stream oder eine neue Sammlung, wobei die Ausgangsdatenquelle unverändert bleibt.
Ausführungsmodell	Lambda-Ausdrücke sind sequentiell und bieten keine integrierte Unterstützung für parallele Verarbeitung.	Streams bieten sowohl sequentielle als auch parallele Verarbeitung. Parallele Streams verwenden eine interne Aufteilung des Workloads, um die Verarbeitung zu beschleunigen.
Operationen	Lambda-Ausdrücke können als einzelne Operation ausgeführt werden und eignen sich daher für kurze, einfache Funktionen.	Streams bieten eine Pipeline von Operationen, die auf eine Datenquelle angewendet wird. Dazu gehören Zwischenoperationen wie <code>filter()</code> , <code>map()</code> , <code>distinct()</code> und Endoperationen wie <code>collect()</code> , <code>forEach()</code> , <code>reduce()</code> .
Verwendung von Intermediären Operationen	Lambda-Ausdrücke selbst sind keine Zwischenoperationen auf einem Stream, sie stellen eine Funktion dar, die direkt auf die Daten angewendet wird.	Streams bieten eine Vielzahl von intermediären Operationen, die auf Daten angewendet werden, und können mit Lazy Evaluation effizient arbeiten.
Lazy Evaluation	Lambda-Ausdrücke haben keine Lazy Evaluation. Sie führen sofort die Logik aus, sobald sie aufgerufen werden.	Streams bieten Lazy Evaluation, bei der Operationen wie <code>filter()</code> oder <code>map()</code> erst dann ausgeführt werden, wenn der Stream tatsächlich konsumiert wird (z. B. mit <code>collect()</code> oder <code>forEach()</code>).

Aspekt	Lambda-Ausdrücke	Streams
Parallele Verarbeitung	Lambda-Ausdrücke selbst bieten keine eingebaute Parallelausführung, aber sie können in parallelen Kontexten verwendet werden (z. B. durch <code>parallelStream()</code> oder durch parallele Threads).	Streams können explizit als parallele Streams verwendet werden (z. B. durch <code>parallelStream()</code> oder durch die <code>parallel()</code> -Methode). Dies nutzt mehrere Prozessoren, um Daten schneller zu verarbeiten.
Fehlerbehandlung	Lambda-Ausdrücke können keine explizite Fehlerbehandlung enthalten. Fehler müssen durch die Methode behandelt werden, in der der Lambda-Ausdruck verwendet wird.	Streams bieten keine spezifische Fehlerbehandlung, aber können durch <code>try-catch</code> -Blöcke innerhalb von Operationen wie <code>map()</code> oder <code>filter()</code> behandelt werden.
Beispiel für einen Lambda-Ausdruck	Beispiel: <code>java (x, y) -> x + y;</code> Dies ist ein Lambda-Ausdruck, der zwei Werte addiert.	Beispiel (mit Stream): <code>java List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5); numbers.stream().map(n -> n * 2).forEach(System.out::println);</code> Hier wird jeder Wert im Stream verdoppelt und ausgegeben.

Wichtige Konzepte

Lambda-Ausdrücke

- Lambda-Ausdrücke bieten eine kompakte Möglichkeit, Funktionen als Parameter zu übergeben oder direkt zu implementieren.

Sie bestehen aus:

- **Parameterliste:** Eine Liste von Eingabeparametern.
- **Arrow-Operator (`->`):** Trennzeichen zwischen Parametern und dem Funktionskörper.
- **Funktionskörper:** Der Code, der die Funktionalität implementiert.
- **Beispiel**

```
(int a, int b) -> a + b;
```

Dieser Lambda-Ausdruck nimmt zwei `int`-Werte und gibt ihre Summe zurück.

Streams

- Ein Stream ist eine Sammlung von Daten, die in einer Pipeline von Funktionen verarbeitet werden können. Ein Stream wird durch eine Sammlung (z.B. List, Set) oder durch `Stream.of()` erstellt.
- Streams bieten eine Reihe von intermediären und terminalen Operationen:
 - Intermediäre Operationen: Verändern den Stream und erzeugen einen neuen Stream (z.B. `filter()`, `map()`).

- Terminale Operationen: Verarbeiten den Stream und liefern ein Endergebnis zurück (z.B. `collect()`, `forEach()`).